Deep Learning

S2026, UG-3

Shaik Mohammad Rafi B

Syllabus:

Unit – 1 : Neural Networks - Introduction of Machine Learning, Linear Classifiers, Perceptron, Multi-class Classification, Non-linear Classification, Neural Networks

Unit – 2 : Multilayer Perceptron - Multilayer Perceptron, Backpropagation; Model Training - Training Aspects of Neural Networks, Hyperparameter Tuning, Gradient Descent Optimization, Regularization, Transfer Learning

Unit – 3 : Convolutional Neural Network - CNN and Architectures

Unit – 4 : Recurrent Neural Network - Word Embeddings, RNN, LSTM, GRU, Attention in RNN

Unit – 5 : Network Visualization and Fooling - Visualization Techniques, Network Fooling and Defense; Generative Neural Network - Autoencoders, Generative Adversarial Network, Variational Autoencoder

Unit – 6 : Advanced Topics and Case Studies - Deep Reinforcement Learning, Self-supervised Learning, Transformers, Neuroscience Approaches, Case Studies

Outcomes

- To gain the fundamentals of deep learning and its uses to tackle **non-linear data**.
- To utilize various **training tricks** for deep learning models.
- To apply the **convolutional neural networks** for processing of **image and video** data for different applications.
- To apply the **recurrent neural networks** to deal with the sequential and time step data for **NLP and Speech** applications.
- To apply the **generative models** to cater the need of sample generation to solve the real world problems.
- To apply the deep learning techniques using recent trends for **real applications using case studies**.

References

Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, The MIT Press, Illustrated edition, 2016.

https://colah.github.io/

datacamp

GeeksforGeeks

Medium

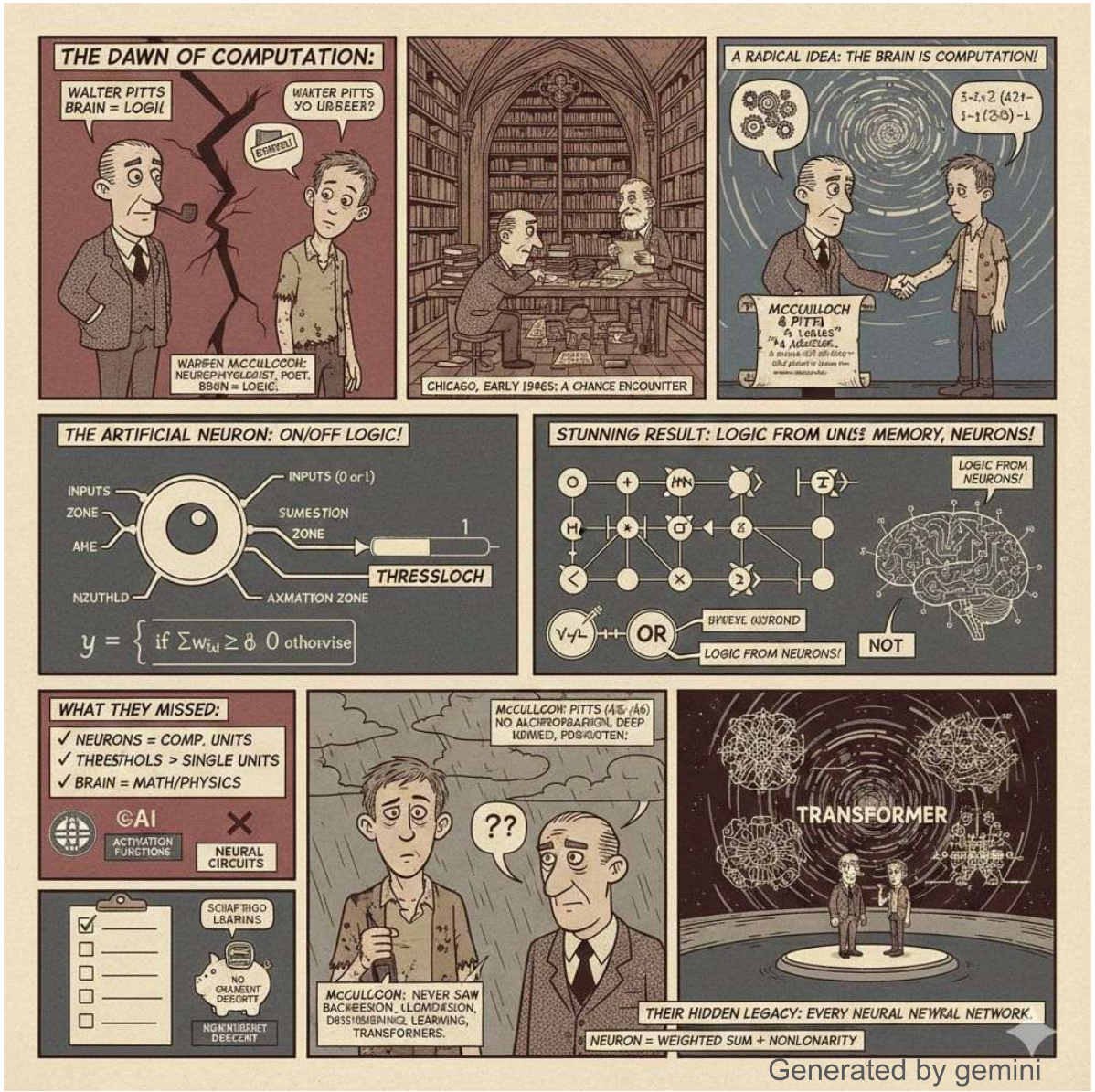
Analytics Vidhya

Analytics India Magazine
Company Profile - Overview

A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY*

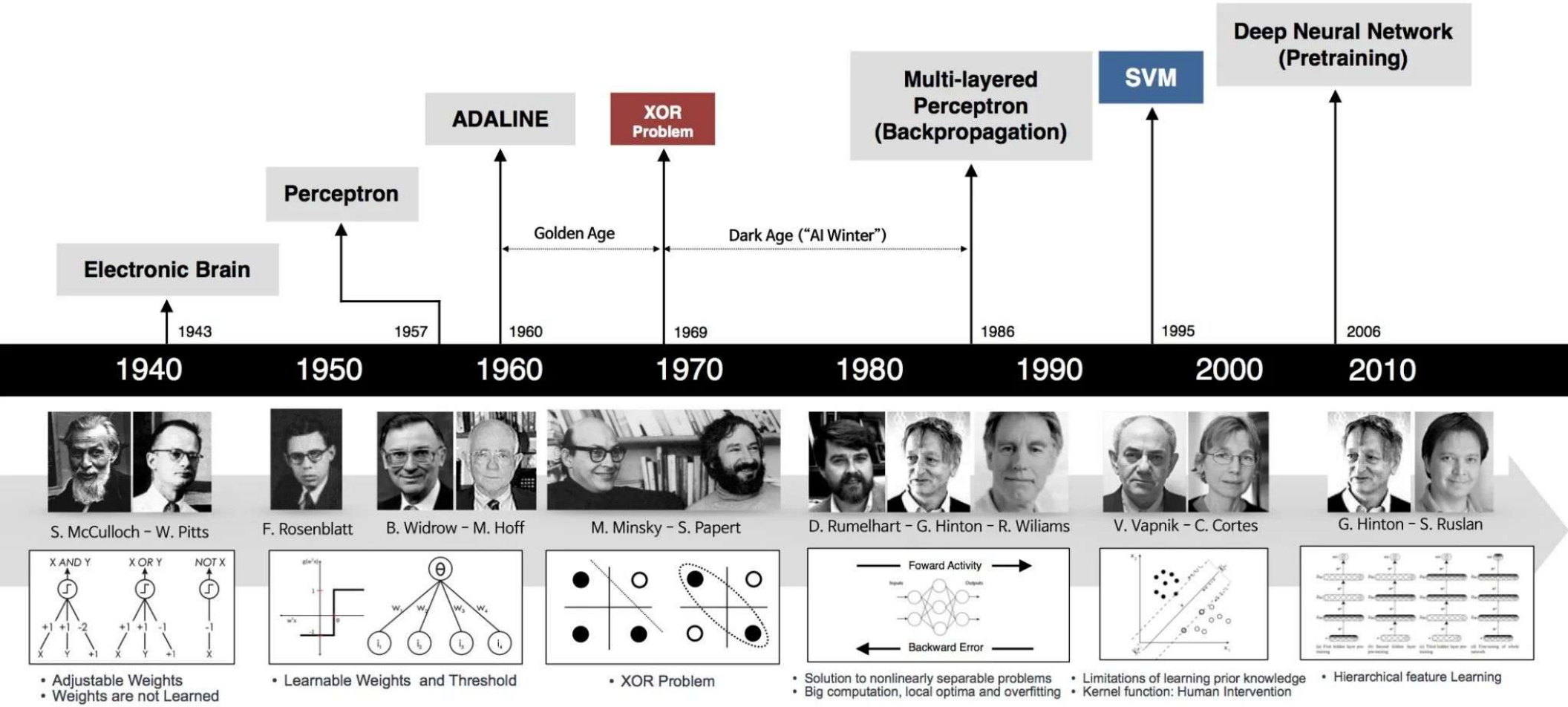
■ WARREN S. MCCULLOCH AND WALTER PITTS
University of Illinois, College of Medicine,
Department of Psychiatry at the Illinois Neuropsychiatric Institute,
University of Chicago, Chicago, U.S.A.

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible neurophysiological assumptions are equivalent, in the sense that for every net behaving under one assumption, there exists another net which behaves under the other and gives the same results, although perhaps not in the same time. Various applications of the calculus are discussed.



Generated by gemini


1940s: Two Unlikely Minds...
"The brain is a logical machine."
Warren McCulloch, Neuroscientist & Philosopher
Walter Pitts, Teenage Math Prodigy
They Meet in Chicago...
What are you working on?
Can the brain do logic?
1943: "A Logical Calculus of Nervous Activity"
What if neurons are like switches?
What if neurons are like switches?
Let's build a brain from math!
Input Signals ∑ w x
y ≡ 1 if ∑ w ≥ θ
Neural Networks can do logic!
These can compute!
AND, OR, NOT...
But... No Learning Yet.
...The weights are fixed.
It can't adapt.
Their Legacy: The Birth of AI
Neuron + Network =
Modern AI
Our idea lives on!
We started it all!



- Adjustable Weights
- Weights are not Learned

- Learnable Weights and Threshold

- XOR Problem

- Solution to nonlinearly separable problems
- Big computation, local optima and overfitting

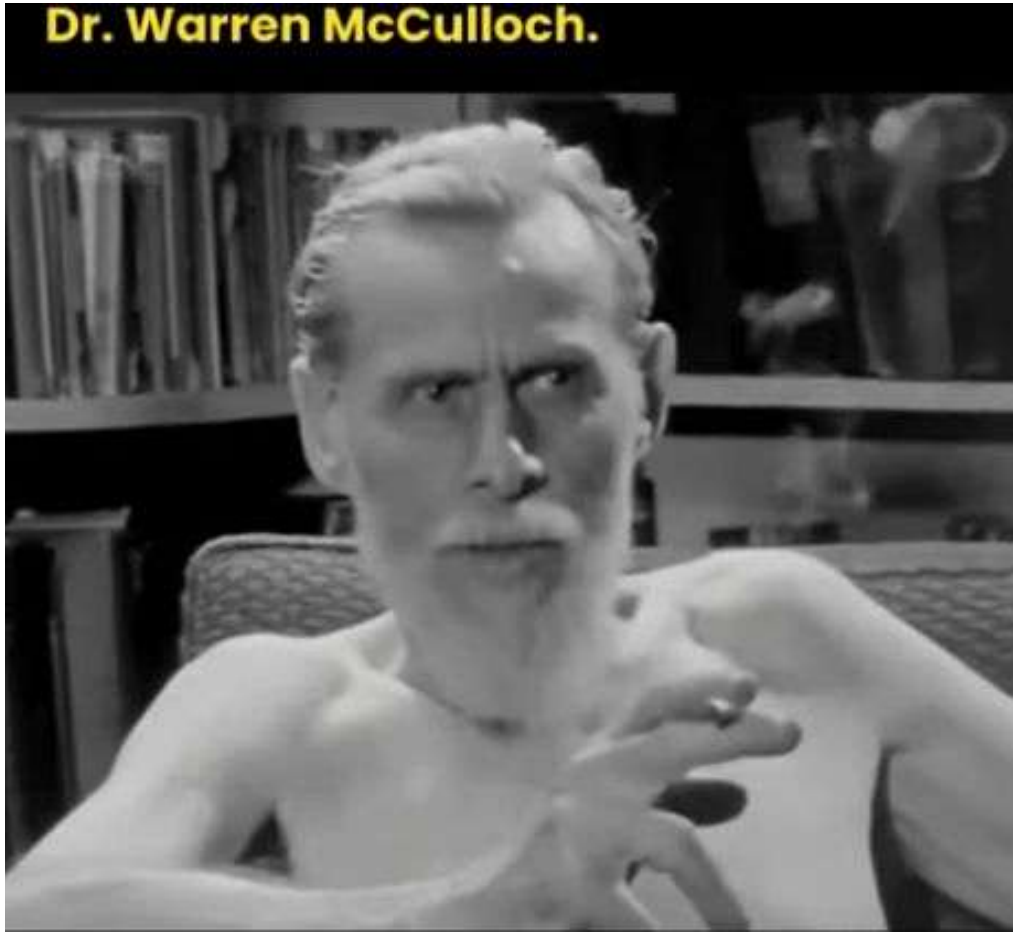
- Limitations of learning prior knowledge
- Kernel function: Human Intervention

- Hierarchical feature Learning

What is a number, that a man may know it,

and a man, that he may know a number?

I am fairly clear about first half of the question and the second half…… not yet!



https://www.youtube.com/shorts/PYNvhsbJSos

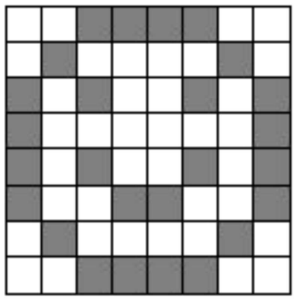
Grading

Type	Marks
MID term test	20
End Sem test	40
Quiz-1 & 2 (scheduled/surprise)	10 (average of 2)
Assignments	10 (average of 10)
Project	15 (2 per team)
Class Participation	5

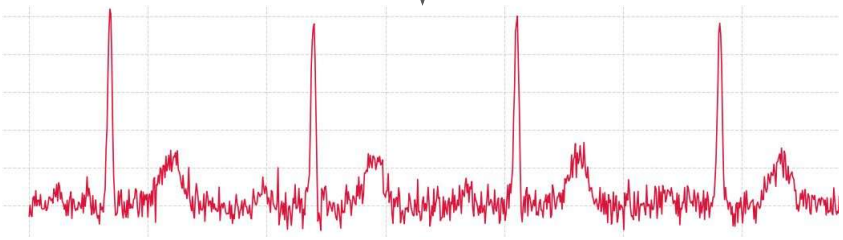
Introduction to Machine Learning

Data for a machine

Pattern for human



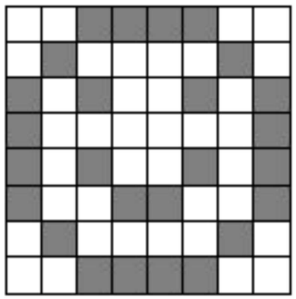
[-0.057, 0.001, -0.052, -0.059, 0.039, 0.047, -0.003, 0.078, -0.002, 0.027,
0.028, -0.002, 0.057, -0.015, -0.019, -0.020, -0.017, 0.025, -0.028, -0.020,
0.008, 0.044, 0.041, 0.025, 0.034, 0.011, 0.087, 0.064, 0.044, 0.111, 0.022,
0.119, 0.101, 0.033, 0.068, 0.019, 0.011, -0.022, 0.004, -0.055,0.027, -0.054,
0.012, 0.044, 0.071, -0.070, 0.034, 0.030, -0.014, -0.028]



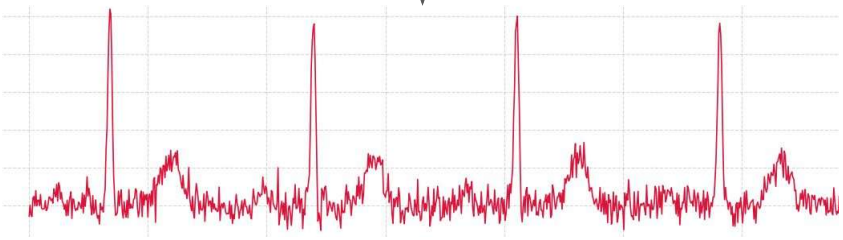
Introduction to Machine Learning

Data for a machine

Pattern for human



[-0.057, 0.001, -0.052, -0.059, 0.039, 0.047, -0.003, 0.078, -0.002, 0.027, 0.028, -0.002, 0.057, -0.015, -0.019, -0.020, -0.017, 0.025, -0.028, -0.020, 0.008, 0.044, 0.041, 0.025, 0.034, 0.011, 0.087, 0.064, 0.044, 0.111, 0.022, 0.119, 0.101, 0.033, 0.068, 0.019, 0.011, -0.022, 0.004, -0.055,0.027, -0.054, 0.012, 0.044, 0.071, -0.070, 0.034, 0.030, -0.014, -0.028]

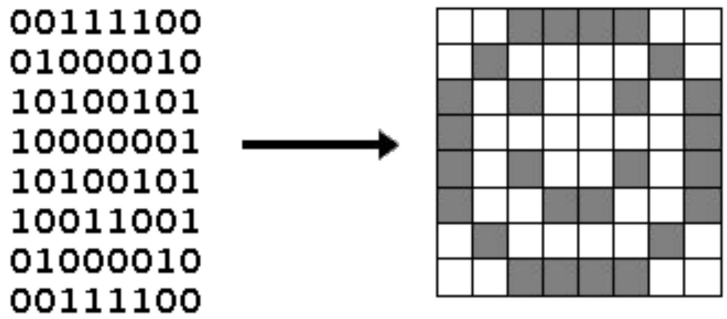


Machine Learning is the process of *finding patterns* in data using algorithms, so that machines can make decisions or predictions without being explicitly programmed.

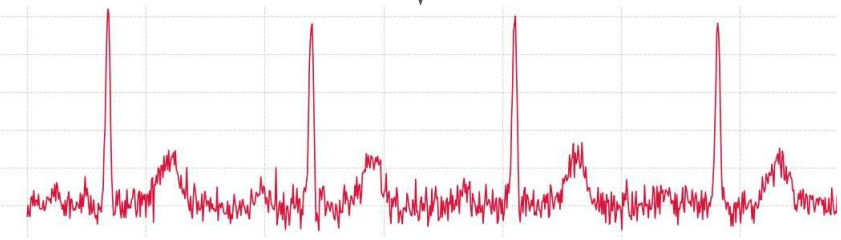
Introduction to Machine Learning

Data for a machine

Pattern for human



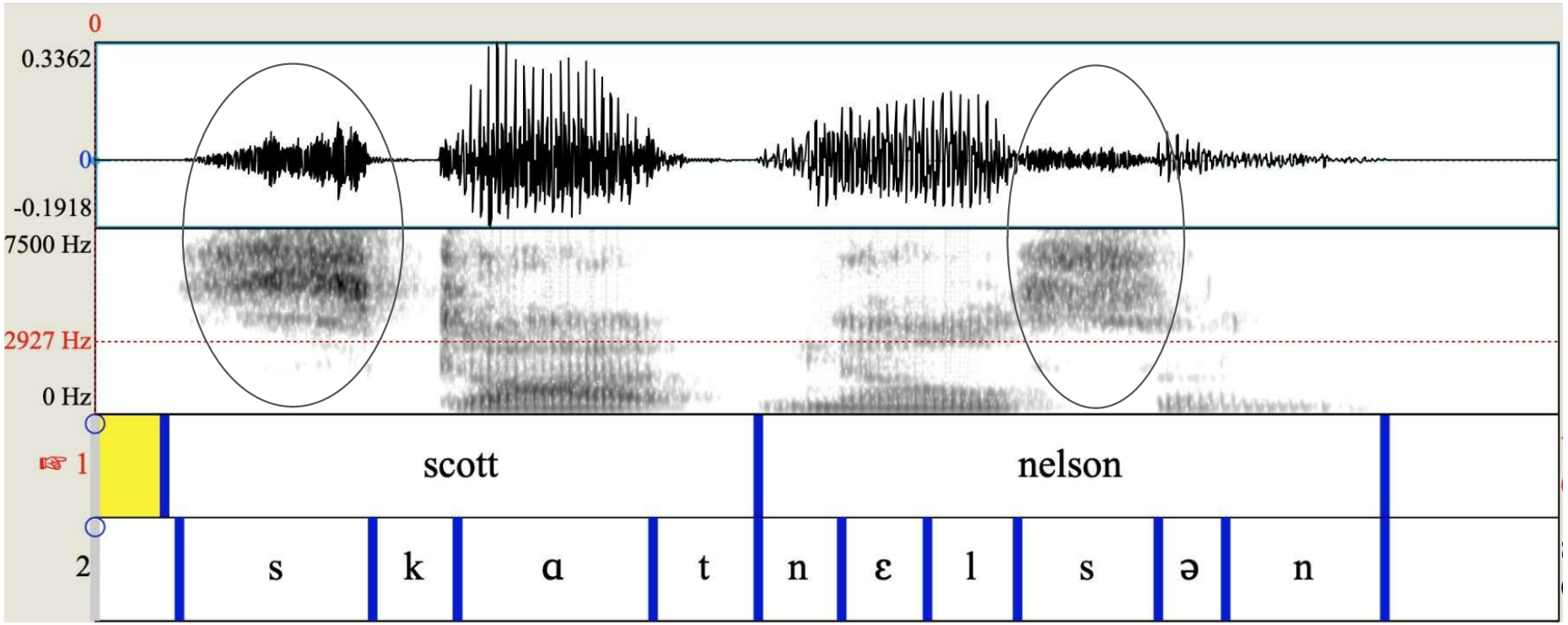
[-0.057, 0.001, -0.052, -0.059, 0.039, 0.047, -0.003, 0.078, -0.002, 0.027, 0.028, -0.002, 0.057, -0.015, -0.019, -0.020, -0.017, 0.025, -0.028, -0.020, 0.008, 0.044, 0.041, 0.025, 0.034, 0.011, 0.087, 0.064, 0.044, 0.111, 0.022, 0.119, 0.101, 0.033, 0.068, 0.019, 0.011, -0.022, 0.004, -0.055,0.027, -0.054, 0.012, 0.044, 0.071, -0.070, 0.034, 0.030, -0.014, -0.028]



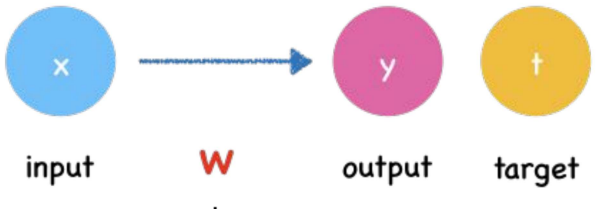
Machine Learning is the process of *finding patterns* in data using algorithms, so that machines can make decisions or predictions without being explicitly programmed.

Machine Learning is pattern recognition + decision making.

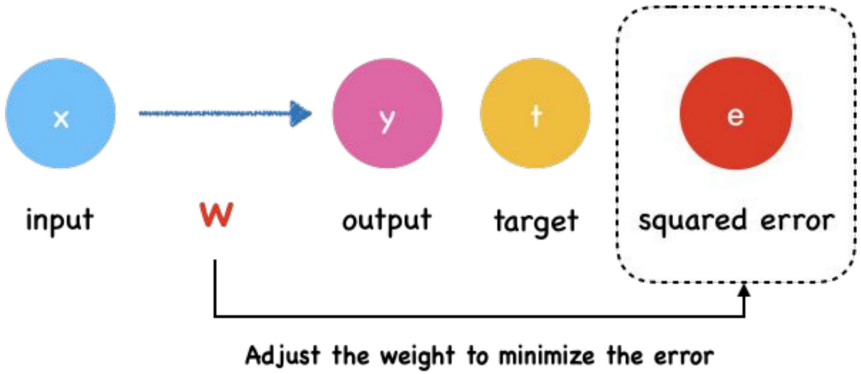
Pattern recognition and Decision making



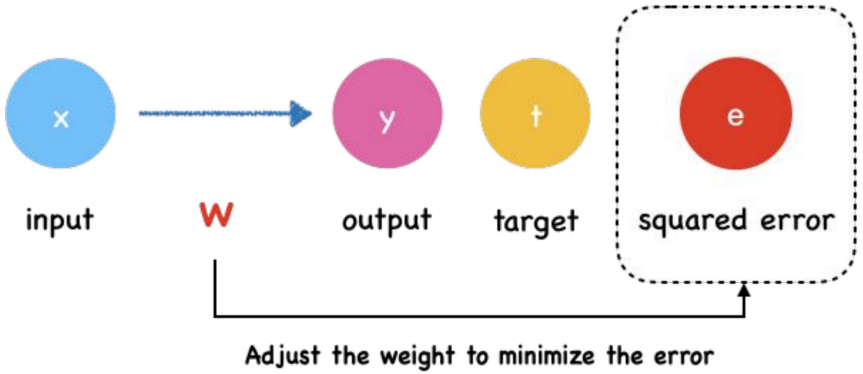
Machine Learning



Machine Learning



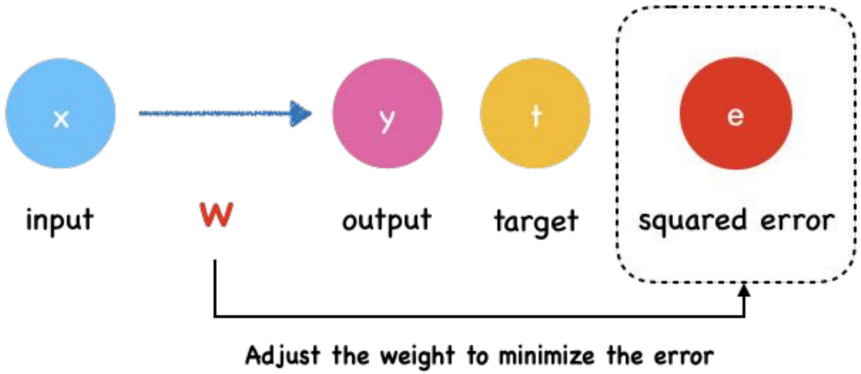
Machine Learning

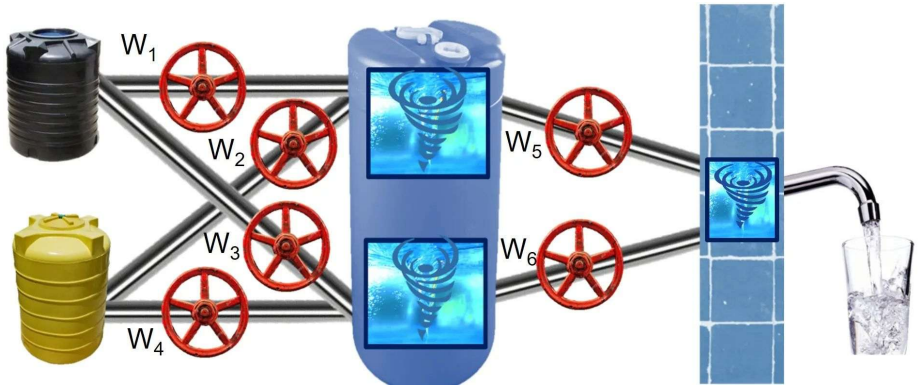


$$(X, Y) : Y \rightarrow f_W(X)$$

The objective of machine learning is to learn a function f that maps input data X to a target output Y, where the function is controlled by learnable parameters W

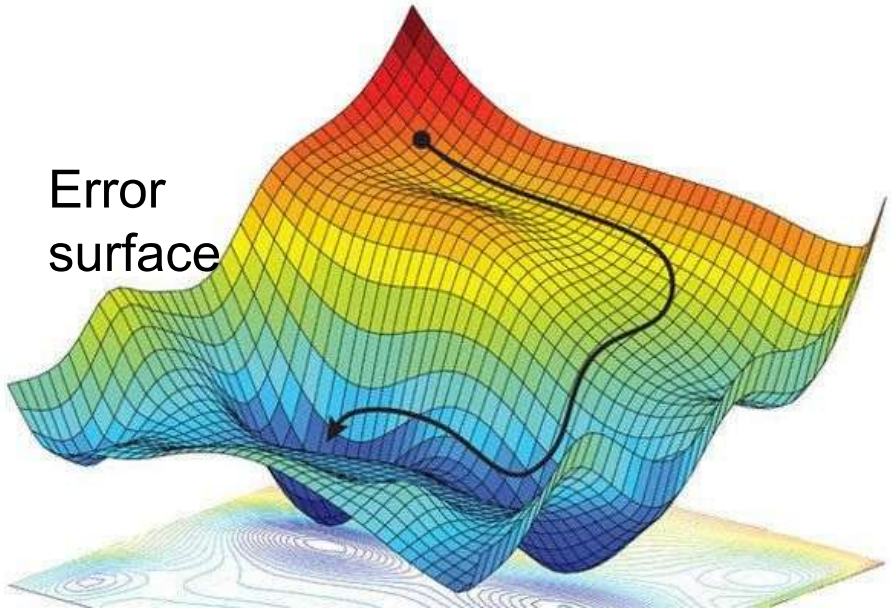
Machine Learning





Tune w to reach optimum solution (minimize error).

$$(X, Y) : Y \rightarrow f_{w}(X)$$



Machine Learning

$$(X, Y) : Y \rightarrow \hat{f}_{W}(X)$$

What type of prediction or analysis? (What is Y)

How do we get the model to learn it? (How to estimate Y)

Machine Learning

$$(X, Y) : Y \rightarrow f_{W}(X)$$

What type of prediction or analysis?

1. Regression
2. Classification

How do we get the model to learn it?

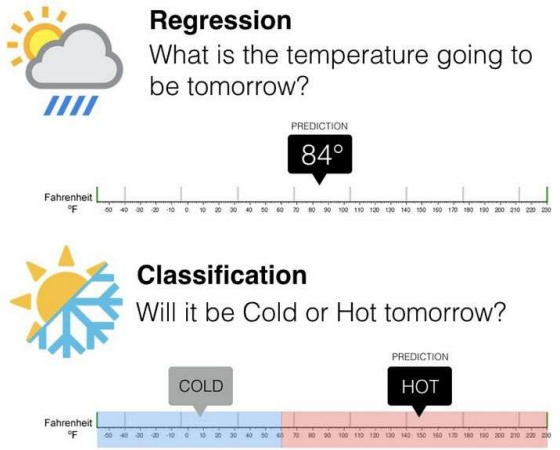
Machine Learning

$$(X, Y) : Y \rightarrow f(X)$$

What type of prediction or analysis?

1. Regression
2. Classification

How do we get the model to learn it?



Machine Learning

$$(X, Y) : Y \rightarrow f_w(X)$$

What type of prediction or analysis?

1. Regression
2. Classification

How do we get the model to learn it?

1. Unsupervised Learning
2. Supervised Learning
3. Reinforcement Learning

Machine Learning

$$(X, Y) : Y \rightarrow f_{w}(X)$$

What type of prediction or analysis?

1. Regression
2. Classification

How do we get the model to learn it?

1. Unsupervised Learning : No labelled data
2. Supervised Learning: Labeled data
3. Reinforcement Learning:

Machine Learning

$$(X, Y) : Y \rightarrow f(X)$$

What type of prediction or analysis?

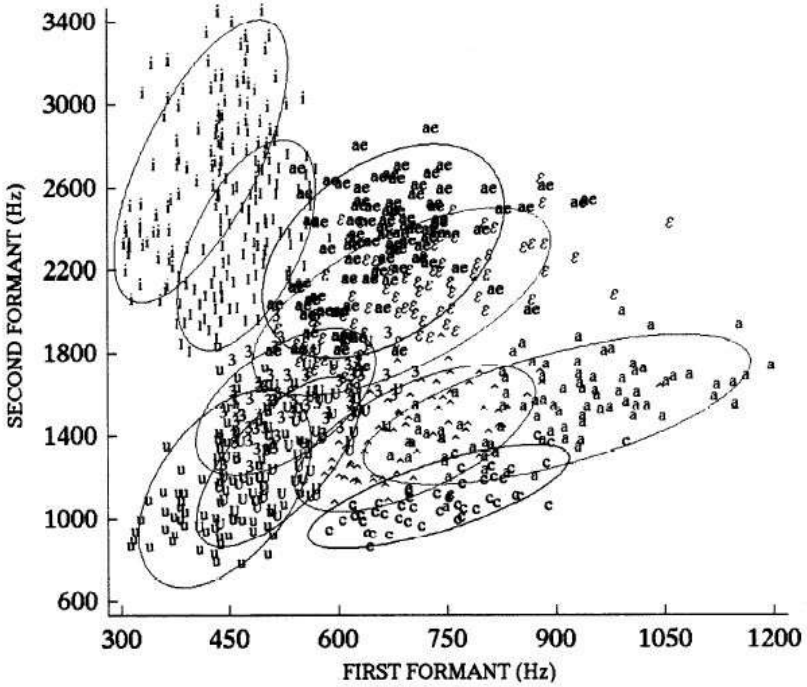
1. Regression
2. Classification

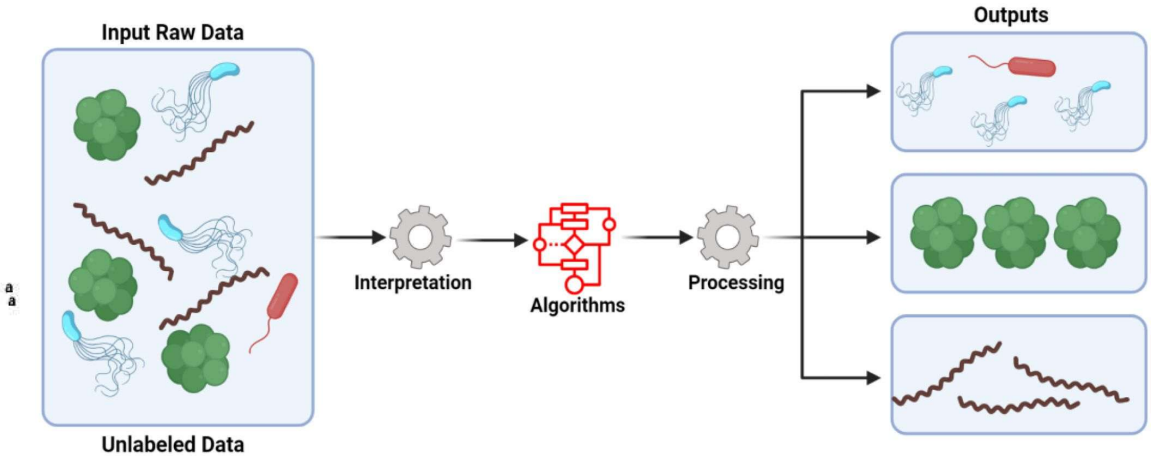
How do we get the model to learn it?

1. Unsupervised Learning
2. Supervised Learning
    a. Fully Supervised: Every data point has corresponding label
    b. Semi-supervised: Exploit unlabelled data and learn from minimally available labelled data
    c. Self-supervised: Pseudo labels
3. Reinforcement Learning

Unsupervised Learning

Raw data→Features→ grouping based on similarities





Semi-supervised: Exploit unlabelled data and learn from minimally available labelled data

Ex: Speech Recognition (Low resource)

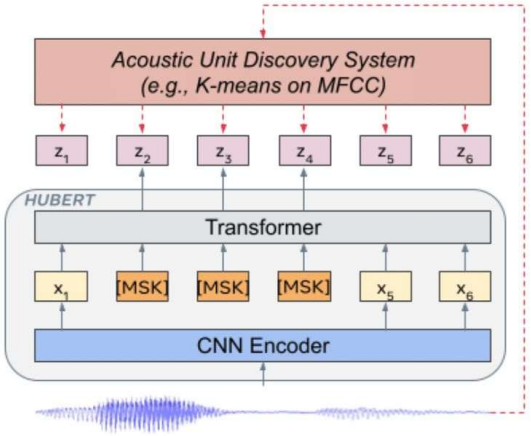
1. Train an initial model using the small labeled speech dataset.
2. Use that model to predict transcriptions for unlabeled audio (these predictions become *pseudo-labels*).
3. Combine labeled and pseudo-labeled data to retrain the model.

Ex: Medical Image Segmentation

1. For each unlabeled medical image, apply two different augmentations (e.g., rotation, noise).
2. Pass both versions through the model and force predictions to be consistent : L_unsup
3. Combine this unsupervised consistency loss with supervised loss from the labeled data: L_sup

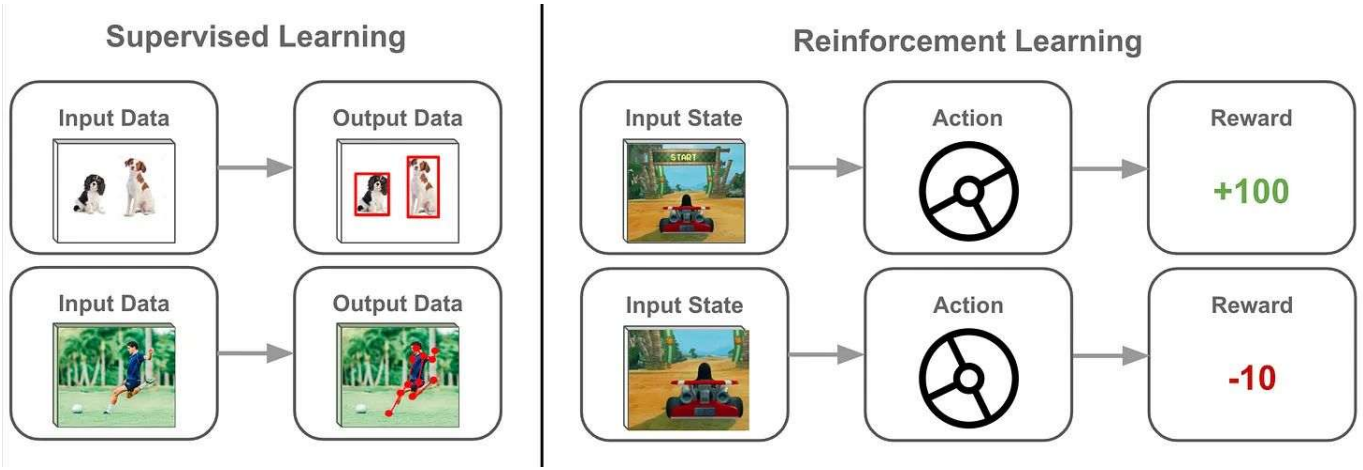
Self-supervised:

Pseudo labels with pretext tasks



Reinforcement learning: states, actions, rewards, transitions, and a policy

Ex: The language model produces some text then receives a score/reward from a human annotator based on the quality of that text.



Regression: Linear Regression

Predicting height based on weight $\hat{h} = f_\theta(w)$

First order $\hat{h} = \theta_1 * w + \theta_0$

Second order $\hat{h} = \theta_2 * w^2 + \theta_1 * w + \theta_0$

- Nonlinear relations between input and labels,
- Linear with respect to parameters θ

Polynomial curve fitting for 1 dimensional data

w(kg)	h (ft)
20	4
45	5
60	6

Regression for multi dimensional data

$$\underline{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ \dots \\ x_n^{(i)} \end{bmatrix} \qquad \underline{\theta} = \begin{bmatrix} \theta_0 \\ \dots \\ \theta_n \end{bmatrix} \qquad \hat{y}^{(i)} = \underline{\theta}^T \underline{x}^{(i)} \qquad X = \begin{bmatrix} x_1^1 & .. & x_n^1 \\ .. & .. & .. \\ x_1^m & .. & x_n^m \end{bmatrix}$$

Regression for multi dimensional data

$$\underline{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ \dots \\ x_n^{(i)} \end{bmatrix} \qquad \underline{\theta} = \begin{bmatrix} \theta_0 \\ \dots \\ \theta_n \end{bmatrix} \qquad \hat{y}^{(i)} = \underline{\theta}^T \underline{x}^{(i)} \qquad X = \begin{bmatrix} x_1^1 & .. & x_n^1 \\ .. & .. & .. \\ x_1^m & .. & x_n^m \end{bmatrix}$$

Objective: minimize MSE

$$J(\theta_0, \dots, \theta_n) = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

$$= \frac{1}{m} \sum_{i=1}^{m} \left(\left(\theta_0 + \theta_1 x_1^{(i)} + \dots + \theta_n x_n^{(i)} \right) - y^{(i)} \right)^2$$

Regression for multi dimensional data

$$\underline{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ \dots \\ x_n^{(i)} \end{bmatrix} \qquad \underline{\theta} = \begin{bmatrix} \theta_0 \\ \dots \\ \theta_n \end{bmatrix} \qquad \hat{y}^{(i)} = \underline{\theta}^T \underline{x}^{(i)} \qquad X = \begin{bmatrix} x_1^1 & .. & x_n^1 \\ .. & .. & .. \\ x_1^m & .. & x_n^m \end{bmatrix}$$

Objective: minimize MSE

$$J(\theta_0, \dots, \theta_n) = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

optimize $\quad \dfrac{\partial J}{\partial \theta_k} = 0$

$$= \frac{1}{m} \sum_{i=1}^{m} \left(\left(\theta_0 + \theta_1 x_1^{(i)} + \dots + \theta_n x_n^{(i)} \right) - y^{(i)} \right)^2$$

Regression for multi dimensional data

$$\underline{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ \dots \\ x_n^{(i)} \end{bmatrix} \qquad \underline{\theta} = \begin{bmatrix} \theta_0 \\ \dots \\ \theta_n \end{bmatrix} \qquad \hat{y}^{(i)} = \underline{\theta}^T \underline{x}^{(i)} \qquad X = \begin{bmatrix} x_1^1 & .. & x_n^1 \\ .. & .. & .. \\ x_1^m & .. & x_n^m \end{bmatrix}$$

Objective: minimize MSE

$$J(\theta_0, \dots, \theta_n) = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

$$= \frac{1}{m} \sum_{i=1}^{m} \left(\left(\theta_0 + \theta_1 x_1^{(i)} + \dots + \theta_n x_n^{(i)} \right) - y^{(i)} \right)^2$$

optimize

$$\frac{\partial J}{\partial \theta_k} = 0$$

$$\underline{\theta}^* = \left[X^T X \right]^{-1} X^T \underline{y}$$

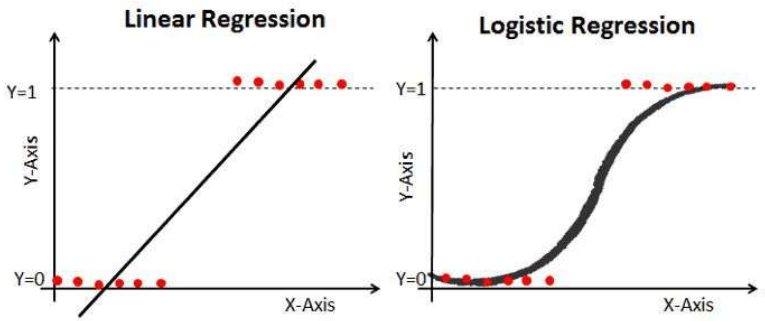
Prediction:

$$\hat{y}_{test} = \underline{\theta}^{*T} \underline{x}_{test}$$

Classification: Logistic Regression

$$\hat{y} = h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}},$$

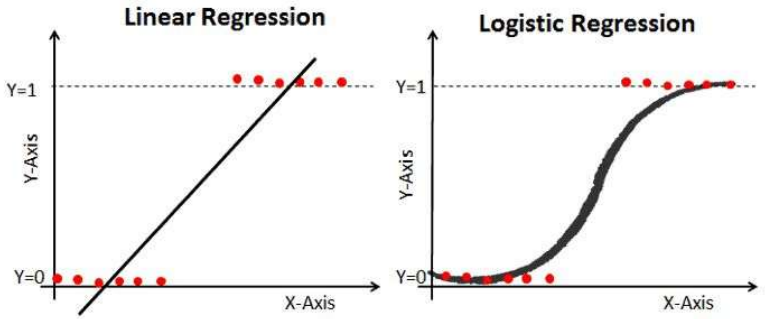
$$g(z) = \frac{1}{1 + e^{-z}}$$



Classification: Logistic Regression

$$\hat{y} = h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}},$$

$$g(z) = \frac{1}{1 + e^{-z}}$$



Logistic regression cost function

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

0 If actual y=1

0 If actual y=0

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

Note: $y = 0$ or 1 always

Gradient Descent for Logistic regression:

$$
\begin{aligned}
\frac{\partial}{\partial \theta_j}\ell(\theta) &= \left(y\frac{1}{g(\theta^T x)} - (1-y)\frac{1}{1-g(\theta^T x)}\right)\frac{\partial}{\partial \theta_j} g(\theta^T x) \\
&= \left(y\frac{1}{g(\theta^T x)} - (1-y)\frac{1}{1-g(\theta^T x)}\right) g(\theta^T x)(1-g(\theta^T x)\frac{\partial}{\partial \theta_j}\theta^T x \\
&= \left(y(1-g(\theta^T x)) - (1-y)g(\theta^T x)\right) x_j \\
&= \left(y - h_\theta(x)\right) x_j
\end{aligned}
$$

Gradient Descent for Logistic regression:

$$
\begin{aligned}
\frac{\partial}{\partial \theta_j}\ell(\theta) &= \left(y\frac{1}{g(\theta^T x)} - (1-y)\frac{1}{1-g(\theta^T x)}\right)\frac{\partial}{\partial \theta_j}g(\theta^T x) \\
&= \left(y\frac{1}{g(\theta^T x)} - (1-y)\frac{1}{1-g(\theta^T x)}\right)g(\theta^T x)(1-g(\theta^T x)\frac{\partial}{\partial \theta_j}\theta^T x \\
&= \left(y(1-g(\theta^T x)) - (1-y)g(\theta^T x)\right)x_j \\
&= \left(y - h_\theta(x)\right)x_j
\end{aligned}
$$

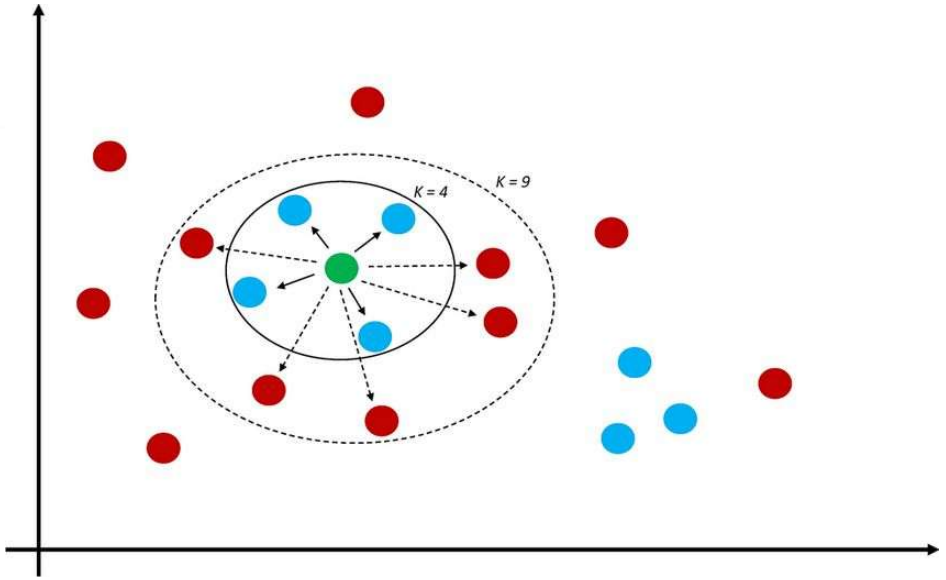
Equating this to zero makes **nonlinearly** dependent on parameters, so you cannot isolate it in closed form → Gradient Descent

$$
\theta_j^{\text{new}} = \theta_j^{\text{old}} - \alpha\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)} \text{ for } j = 0, 1, \ldots, n
$$

K-NN: A distance based classifier

Classify $(\mathbf{X}, \mathbf{Y}, x)$ // $\mathbf{X}$: training data, $\mathbf{Y}$: class labels of $\mathbf{X}$, x: unknown sample
for $i = 1$ **to** m **do**
  Compute distance $d(\mathbf{X}_i, x)$
end for
Compute set I containing indices for the k smallest distances $d(\mathbf{X}_i, x)$.
return majority label for $\{\mathbf{Y}_i$ where $i \in I\}$

A **lazy learning** algorithm: doesn't have an explicit training phase



K-means Clustering:

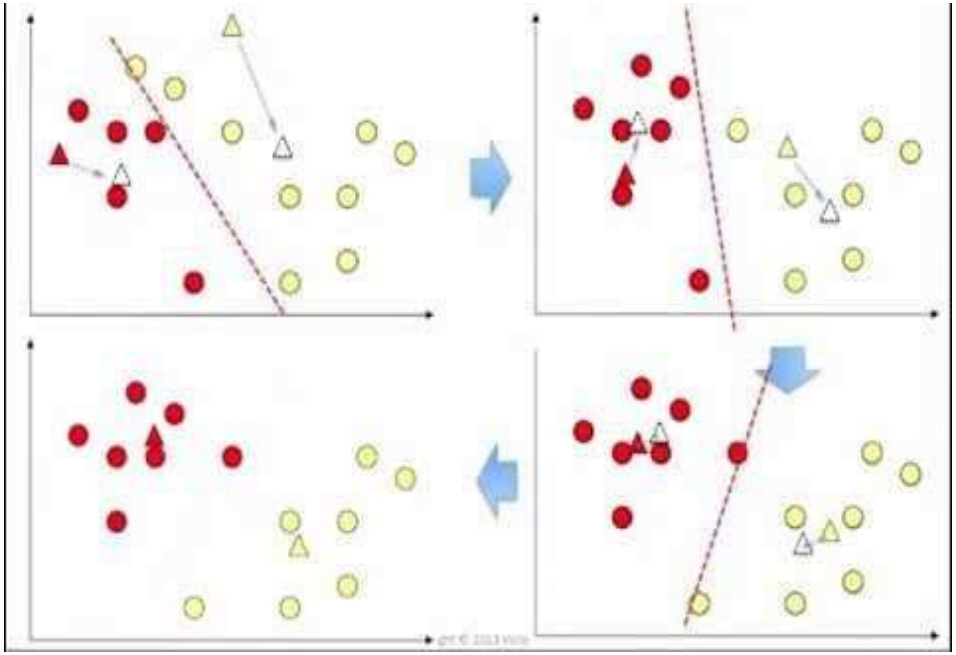
1. **Initialize**: Choose K random points as centroids.
2. **Assign Step**: Assign each data point to the nearest centroid (based on distance, e.g., Euclidean).

$$c_i = \arg\min_{k \in \{1,\ldots,K\}} \|x_i - \mu_k\|^2$$

3. **Update Step**: For each cluster, update the centroid as the mean of its assigned points.

$$\mu_k = \frac{1}{N_k} \sum_{i:c_i=k} x_i$$

4. **Repeat** steps 2–3 until centroids do not change (convergence) or a max iteration limit is reached.



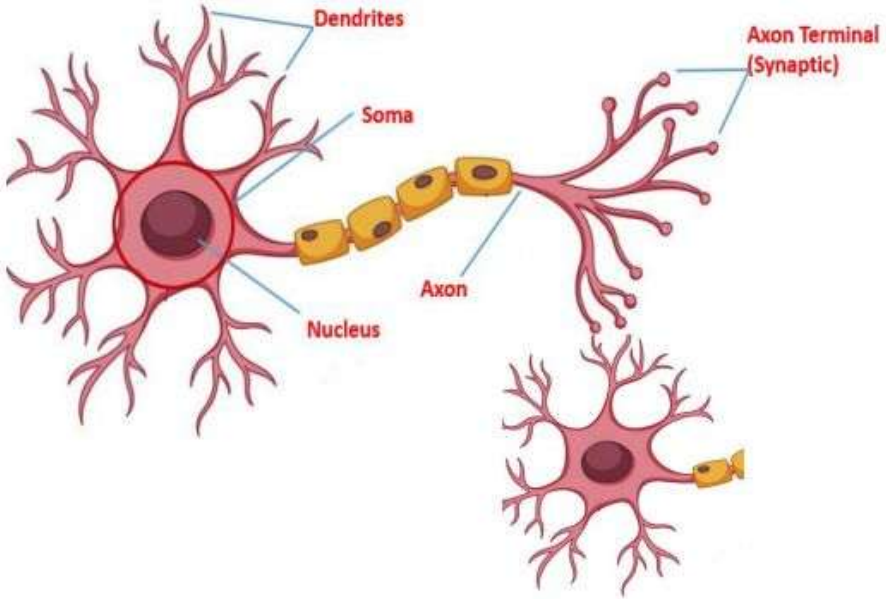
Assignment-1

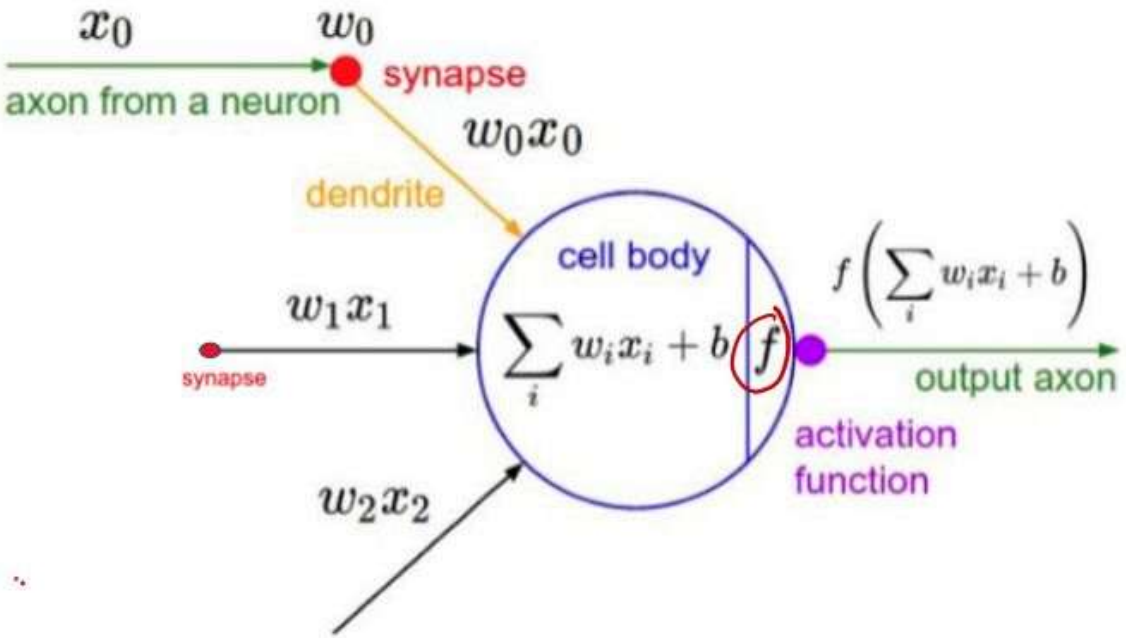
Write Python Programs for implementing

1. Linear regression: calculate brain weight based on head size
    a. Load data (X,Y): 40 number of data points
    b. Get data matrix
    c. Find w as derived
    d. Predict labels
2. Logistic regression: classify gender based on head size and brain weight
    a. Load data: all data points
    b. Initialize random weights, Calculate gradient
    c. Update weights through gradient descent
    d. Predict

Get data from here
https://drive.google.com/file/d/1rAP-MQFUi8m8nY7RXx5pRZrds8umhCwm/view?usp=sharing

Artificial Neuron

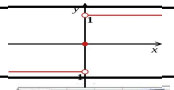
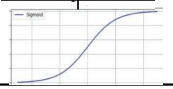
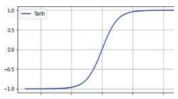
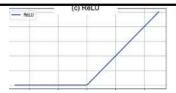
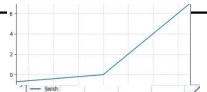
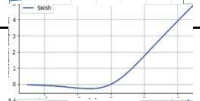
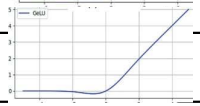




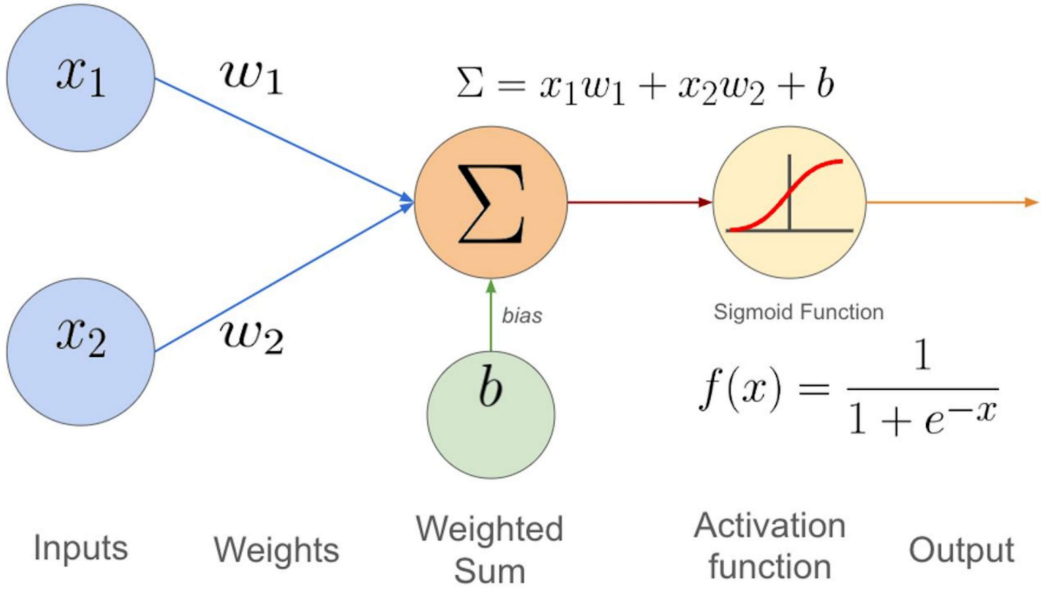
Cell nucleus (Soma)	Node
Dendrites	Input
Synapse	Weights
Axon	Output

Activation Functions

- Decides neuron firing strength
- Introduce non-linearity

Type	Mathematical Equation	Usage (layers)	Limitations
Sign (Theory)	$(\text{sign}(x) \in -1, 0, +1)$ 	Hard binary decision (out)	Non-differentiable, no gradient
Sigmoid	$(\sigma(x) = \frac{1}{1+e^{-x}})$ 	Probabilistic binary decision (out)	Vanishing gradient, not zero-centered
Tanh	$(\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}})$ 	Bipolar binary decision (hidden, out)	Saturation (-1,1) & vanishing gradient (when used as hidden layer AF)
Softmax	$(\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}})$	Multi-class probabilistic decision (out)	Sensitive to scale (softmax values of z vs 5z changes drastically)
Linear	$(f(x) = x)$	Final projection or regression (out)	No non-linearity
ReLU	$(\max(0, x))$ 	Preserve positive values, avoid vanishing gradient (hidden)	Dying ReLU (zero gradient for negatives)
Leaky ReLU	$(\max(\alpha x, x))$ 	Prevent dying ReLU, allow negative gradient (hidden)	Extra hyperparameter, slight bias
Swish	$(x \cdot \sigma(x))$ 	Smooth non-linearity, better gradient flow (hidden)	Computationally expensive
GELU	$(x \cdot \Phi(x))$ 	Probabilistic gating (hidden)	Costly, complex

Perceptron (The logistic regression)



Probabilistic Analysis of binary classification

$$P(y=1 \mid x; w) = \sigma(z) = \frac{1}{1+e^{-z}}, \quad z = w^T x$$

$$P(y=0 \mid x; w) = 1 - \sigma(w^T x)$$

Odds= p/(1-p)

- Odds = 1 → event equally likely to happen or not
- Odds > 1 → event more likely to happen
- Odds < 1 → event less likely to happen

$$P(y=1 \mid x; w) = \sigma(z) = \frac{1}{1+e^{-z}}, \quad z = w^T x$$

Simplifying

$$\log \frac{P(y=1 \mid x)}{1 - P(y=1 \mid x)} = w^T x$$ Denotes log of odds

Since y ∈ {0,1} , the target follows a **Bernoulli distribution**:

Probability Mass function (likelihood):

p(y | x;w) = p if y=1

= q = 1-p if y=0

Rewritten as

$$P(y \mid x; w) = p^y (1-p)^{1-y}$$

For N data points $\{(x_i, y_i)\}_{i=1}^{N}$

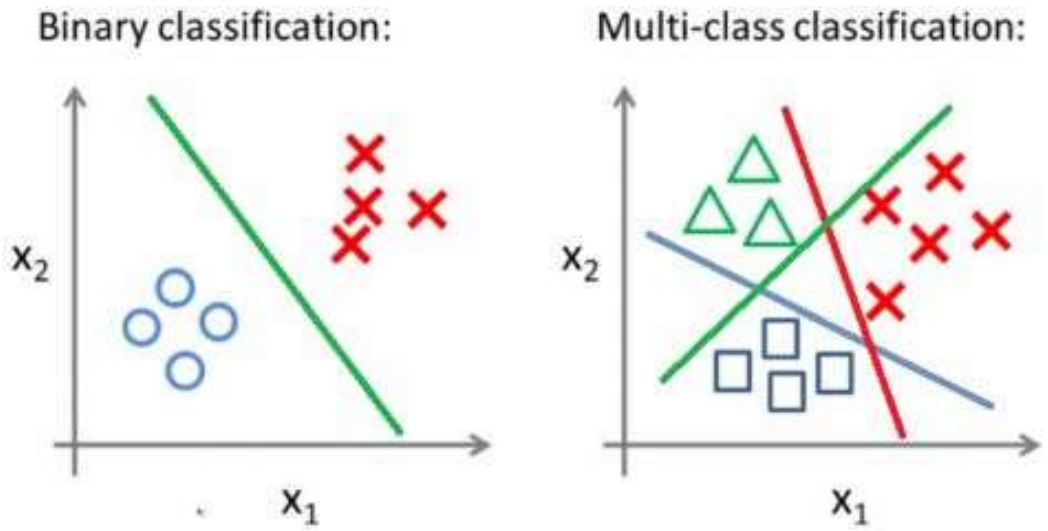
Assuming independence for all i, the likelihood becomes

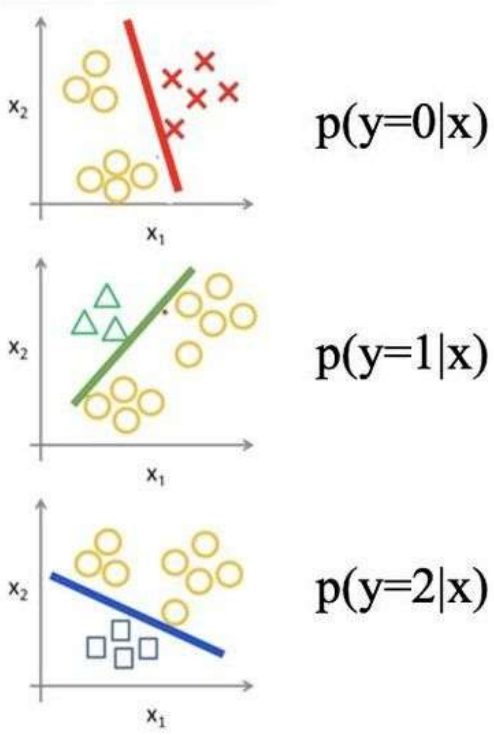
$$\mathcal{L}(w) = \prod_{i=1}^{N} P(y_i \mid x_i; w) = \prod_{i=1}^{N} \sigma(z_i)^{y_i} (1-\sigma(z_i))^{1-y_i}$$

Log likelihood (taking log)

$$\log \mathcal{L}(w) = \sum_{i=1}^{N} [y_i \log \sigma(z_i) + (1-y_i) \log(1-\sigma(z_i))]$$

Multiclass classification





One vs all

One-hot encoding

Pet type	Pet Code
cats	0
dogs	1
birds	2

categorical data

Pet type	is cat	is dog	is bird
cats	1	0	0
dogs	0	1	0
birds	0	0	1

one hot encoding

Labels are scalars

Labels are vectors

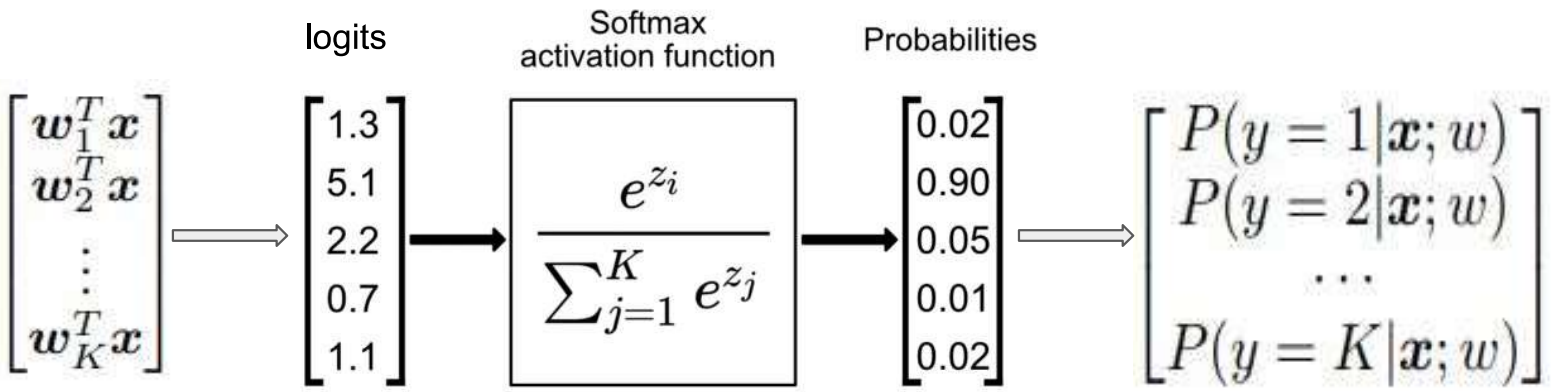
n classes => n number of n dimensional one-hot encodings

Binary classification: one weight vector (parameters)

Prediction: $y_{1x1} = W_{1xd} * X_{dx1} \rightarrow$ sigmoid p(y) $\rightarrow$ Threshold $\rightarrow$ class 0 or 1

K-class (multi) classification: K number of weight vectors

Prediction: $Y_{Kx1} = W_{Kxd} * X_{dx1} \rightarrow$ softmax p(Y) $\rightarrow$ argmax $\rightarrow$ class 1 or 2 or … K



sum=1

Multiclass generalization of the Bernoulli distribution.

$$P(Y = k) = p_k,$$

Probability Mass Function

$$P(y) = \prod_{k=1}^{K} p_k^{y_k}$$

Negative log likelihood

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log p_k$$

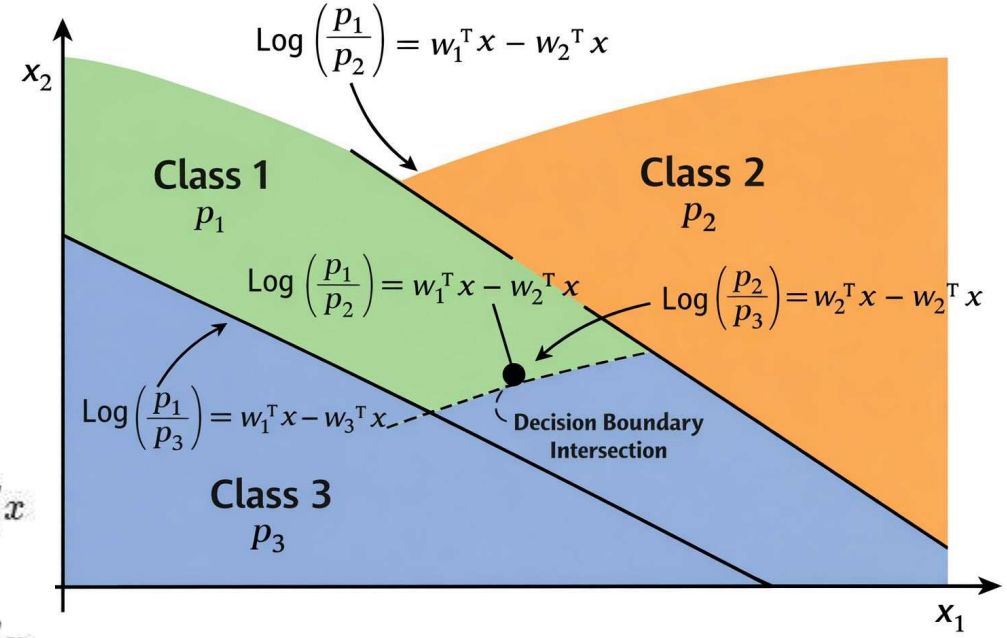
For N independent samples

$$J = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \log p_{ik} \qquad \text{where} \quad p_{ik} = \frac{e^{w_k^T x_i}}{\sum_{j=1}^{K} e^{w_j^T x_i}}$$

Log of odds

$$\log \frac{p_k}{p_j} = w_k^T x - w_j^T x$$

This indicates boundary where pk=pj



Green region (Class 1):

$$w_1^T x > w_2^T x \text{ and } w_1^T x > w_3^T x$$

Orange region (Class 2):

$$w_2^T x > w_1^T x \text{ and } w_2^T x > w_3^T x$$

Blue region (Class 3):

$$w_3^T x > w_1^T x \text{ and } w_3^T x > w_2^T x$$

Show that binary classification is a case of multiclass classification with K=2